

Neuro-Symbolic SKOS Alignment

A Technical Planning Document

Hackathon Edition · May 2026

Overview

This document summarises a technical planning session for a hackathon project targeting automated, semantically grounded alignment of SKOS controlled vocabularies. It covers the problem domain, the three core tasks, the data available, and a layered technical approach combining neural embeddings, Graph Neural Networks, symbolic inference, and established frameworks (GSAS, AMT). A realistic two-day execution plan is included.

1 • Problem Field

1.1 Knowledge Organization Systems and SKOS

Knowledge Organization Systems (KOS) — thesauri, classification schemes, authority files — are the backbone of semantic interoperability in libraries, archives, and cultural heritage institutions. SKOS (Simple Knowledge Organization System) is the W3C standard for expressing these systems as RDF graphs. A SKOS concept carries labels (prefLabel, altLabel), definitions, scope notes, and hierarchical/associative relations (broader, narrower, related).

SKOS also defines a family of **mapping properties** for expressing correspondences between concepts in *different* vocabularies: exactMatch, closeMatch, narrowMatch, broadMatch, and relatedMatch. These six relation types have well-defined logical semantics — exactMatch is symmetric and transitive, narrowMatch is asymmetric, etc. — making them amenable to formal reasoning.

1.2 The Reconciliation Gap

Current reconciliation practice — matching concepts across vocabularies — relies heavily on **string similarity**: if two concept labels match exactly or near-exactly, a mapping is asserted. This approach has three fundamental failure modes:

- **False negatives**: synonyms, cross-language equivalents, and notational variants are missed entirely.
- **False positives**: homonyms (e.g. 'Krebs' = cancer OR crustacean) are matched without disambiguation.
- **Type poverty**: all matches are asserted as exactMatch regardless of the actual semantic relationship.

1.3 The Data Situation

The coli-conc project (GBV) provides a JSKOS API with over **1.1 million mappings** across 113 vocabularies, created by multiple human curators. This dataset is unique in that:

- The same concept pair is often mapped by multiple curators independently, creating a **disagreement signal** — a measure of mapping certainty.
- Mappings span the full SKOS relation type spectrum, not just exactMatch.
- Annotations (bodyValue +1 / -1) provide explicit quality assessments.
- Concepts carry rich metadata: prefLabel, altLabel, definition, scopeNote, broader/narrower/related, inScheme.

Core claim: this dataset is large enough and structured enough to train a domain-specific alignment model that substantially outperforms string reconciliation — and to evaluate that improvement rigorously.

2 · Specific Tasks

Three concrete tasks define the scope of the project. All three reduce to the same core operation: computing semantic similarity between concepts, possibly within a graph-structural context.

Task	Input	Output	Core operation
1 · Thesaurus from scratch	Flat array of terms	SKOS thesaurus with broader/narrower hierarchy	Cluster terms by semantic similarity; label clusters; serialize as RDF
2 · Insert into existing thesaurus	New terms + existing SKOS vocabulary	Terms attached at the correct position in the hierarchy	Find nearest existing concept; determine correct depth (sibling vs. child)
3 · Map n thesauri	Two or more SKOS vocabularies	Typed SKOS mappings with confidence scores	Cross-vocabulary nearest-neighbor retrieval + relation type classification

Key insight: Tasks 1–3 share a single core capability — a good semantic similarity function between concept representations. Build it once; apply it three ways.

3 · Data Pipeline

3.1 Fetching from the JSKOS API

All data is fetched from <https://coli-conc.gbv.de/api/> using the JSKOS REST API. Mappings are streamed as NDJSON (one JSON object per line) to avoid memory issues at 1.1M records. Vocabularies and concordances are fetched via paginated JSON.

Outputs:

- data/mappings.ndjson — all mappings, ~1.1M records
- data/vocabularies.json — 113 concept schemes with metadata
- data/concordances.json — 113 concordances
- data/edge_list.tsv — flat edge list: from_uri, to_uri, type, concordance
- data/concepts/.ndjson — per-vocabulary concept files (optional)

3.2 Aggregating Multiple Mappings

Because multiple curators map the same concept pairs, the raw edge list contains duplicate (from, to) pairs with different relation types. These are aggregated into a single confident edge per pair:

dominant_type — majority-vote relation type

confidence = agreement ratio (1.0 = unanimous, 0.33 = one of three types)

n_mappings = number of independent source mappings

n_concordances = number of distinct concordances (cross-concordance = stronger signal)

Training signal quality tiers:

Tier	Filter	Use
Ground truth	confidence $\geq$ 0.8, n_concordances $\geq$ 2	Test set, hard positives
Strong signal	confidence $\geq$ 0.66, n_mappings $\geq$ 2	Primary training data
Weak signal	confidence $<$ 0.66, n_mappings $\geq$ 3	Ambiguous cases — hard negatives
Contested	high n_mappings, low confidence	Uncertainty output — flag to curator

3.3 Concept Representation

Rather than encoding only the prefLabel, each concept is represented as a **structured text document** that captures all available semantic information:

- **prefLabel** — primary identifier, highest weight
- **altLabel** — synonyms and variants; critical for synonym recall
- **definition / scopeNote** — disambiguates homonyms
- **broader label** — contextualises within hierarchy
- **narrower labels (top 5)** — act as implicit definition
- **inScheme** — vocabulary membership (categorical feature)

Properties **not** used as input features: broadMatch, narrowMatch, exactMatch etc. These are the mapping relations being predicted — using them as input is target leakage. Exception: mappings to a *third* vocabulary can serve as transitivity hints.

4 · Computing Hierarchical Relations from Embeddings

Cosine similarity is **symmetric** — $\text{cosine}(A,B) = \text{cosine}(B,A)$ — but hierarchy is not. 'Leukozyten broader Blut' differs from 'Blut broader Leukozyten'. Three approaches address this, ranging from practical to theoretically elegant.

4.1 Asymmetric Projection Heads (recommended for hackathon)

Keep standard euclidean embeddings from the transformer. Add two learned linear projections: one for 'I am the broader concept', one for 'I am the narrower concept'. The model learns that the `as_broader` projection clusters general concepts and `as_narrower` clusters specific ones:

```
narrowMatch(A,B) = cosine( as_narrower(emb_A), as_broader(emb_B) )
broadMatch(A,B) = cosine( as_broader(emb_A), as_narrower(emb_B) )
exactMatch(A,B) = cosine( emb_A, emb_B ) [symmetric, no projection needed]
```

Training signal: intra-vocabulary broader/narrower edges from the concept files give asymmetric pairs. Implementation is ~30 lines and ~2 hours.

4.2 Order Embeddings / Entailment Cones (most principled)

Each concept is mapped to a **cone** in embedding space. A is broader than B if B's cone is contained within A's cone — directly encoding the partial order semantics of `skos:broader`. A violation loss penalises pairs where B is not dominated coordinate-wise by A:

```
order_violation(A, B) = sum( relu(emb_B - emb_A)^2 ) [zero = A correctly broader than B]
```

Why this fits SKOS: `skos:broader/narrower` is a partial order relation. Order embeddings are designed for partial orders — the semantics match the math directly.

4.3 Hyperbolic Embeddings — Poincaré Space (geometrically correct)

Hyperbolic space has negative curvature and expands exponentially outward, naturally mirroring how concept trees branch. **Broader concepts sit near the origin; narrower concepts near the boundary.** Both similarity and direction are encoded in a single geometry:

- $\|emb_A\| < \|emb_B\| \rightarrow A$ is likely broader than B (norm encodes specificity)
- Library: `geoopt` (Poincaré Ball manifold). Numerically delicate — post-hackathon upgrade.

4.4 What Flat Similarity Can Still Tell You

Even without directional encoding, standard embeddings provide useful hierarchy signals:

Neighborhood density: broader concepts have more nearby neighbors (narrower children cluster around them). Count neighbors above a cosine threshold — higher count suggests broader concept.

Directional implication: a narrower concept's text implies its broader context but not vice versa. A cross-encoder trained on asymmetric (A, B, `narrowMatch`) vs (B, A, `narrowMatch`) pairs learns this without any geometric machinery.

Relation	How to compute	Sym?	Best approach
<code>exactMatch</code>	High cosine similarity > 0.90	Yes	Bi-encoder

closeMatch	Cosine similarity 0.75–0.90	Yes	Bi-encoder
relatedMatch	Cosine 0.55–0.75 + shared broader concept	Yes	Bi-encoder + graph rule
narrowMatch	Asymmetric head score + ancestral inference rule	No	Proj. heads + symbolic
broadMatch	Inverse of narrowMatch	No	Proj. heads + symbolic
broader/narrower (intra-voc)	Order embeddings or Poincare	No	Hyperbolic (post-hackathon)

Key takeaway: similarity handles exact/close/related well. Hierarchy requires asymmetric mechanisms. For the hackathon: asymmetric projection heads (4.1) combined with symbolic ancestral rules give the best effort/payoff ratio — the rules bootstrap what the neural head has not learned yet.

5 · Neural Approaches

5.1 Bi-encoder (Retrieval)

A multilingual sentence transformer (e.g. **intfloat/multilingual-e5-base** or **BAAI/bge-m3**) encodes each concept-as-document into a dense vector. Candidate retrieval uses FAISS approximate nearest-neighbor search. This replaces the binary string-match with a soft ranked list.

Advantage: handles synonyms, cross-language, encoding is done once.

Limitation: no interaction between concepts during encoding; cannot distinguish exactMatch from closeMatch.

5.2 Cross-encoder (Reranking)

A cross-encoder sees both concepts simultaneously as a single sequence: [concept_A text] [SEP] [concept_B text]. Full attention between the two enables fine-grained relation type classification. Fine-tuned on JSKOS mapping pairs with confidence weighting. Used to rerank the top-k bi-encoder candidates.

Starting model: cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 (multilingual, fast).

5.3 Asymmetric Projection Heads

Standard cosine similarity is symmetric — it cannot distinguish A narrowMatch B from B narrowMatch A. Two separate learned projection heads on top of the shared encoder produce asymmetric scores for directed relations:

- $\text{broader_proj}(A) \cdot \text{narrower_proj}(B) \rightarrow \text{narrowMatch score}$
- $\text{narrower_proj}(A) \cdot \text{broader_proj}(B) \rightarrow \text{broadMatch score}$
- $\text{cosine}(\text{emb_A}, \text{emb_B}) \rightarrow \text{exactMatch / closeMatch score (symmetric)}$

5.4 Graph Neural Network (GNN)

A concept's identity is not just its label — it is its *position in a web of connections*. A GNN operating over the mapping graph learns embeddings that encode this neighborhood, enabling disambiguation (same label, different graph context = different embedding) and zero-shot inference for unmapped concepts via their neighbors.

Architecture:

- **Node features:** text embeddings from bi-encoder (warm start)
- **Edge types:** one per SKOS relation (HeteroData graph)
- **Model:** HeteroConv with SAGEConv per relation type $\rightarrow$ GraphSAGE baseline, HGT for best results
- **Training:** NeighborLoader mini-batches (tractable on 12 GB GPU)
- **Task:** link prediction — positive pairs from high-confidence mappings, negatives sampled randomly

GNN vs. pure text: GNN wins on ambiguous concepts (same label, different domain). The key benchmark is performance on homonyms and cross-language pairs where string matching fails entirely.

5.5 Fine-tuning Strategy

The 1.1M JSKOS mapping pairs are used to fine-tune the sentence transformer with confidence-weighted cosine similarity loss. One epoch over 100k pairs takes ~20 minutes on CPU, producing a domain-specific embedding model that outperforms the generic multilingual baseline on KOS terminology.

Training pairs:

- exactMatch → label 1.0 (strong positive)
- closeMatch → label 0.7
- narrowMatch / broadMatch → label 0.4 (related but asymmetric)
- Random non-mapped pairs → label 0.0 (negative)
- Weight each pair by confidence score from aggregation step

6 · Symbolic Inference

Neural models learn statistical patterns but have no knowledge of SKOS semantics. Symbolic rules encode what SKOS *means* — and can both fill gaps in sparse mapping coverage and correct logically inconsistent neural outputs.

5.1 Inference Rules

Rule	Pattern	Inferred	Confidence decay
1 · Ancestral inheritance	A broader B; B exactMatch D	A narrowMatch candidates_of(D)	× 0.70 per hop
2 · Sibling triangulation	A broader P; B broader P; A exactMatch C	B closeMatch/narrowMatch siblings_of(C)	× 0.65
3 · Transitivity chain	A exactMatch B (voc1→voc2); B exactMatch C (voc2→voc3)	A exactMatch C (voc1→voc3)	× 0.90 per hop
4 · Cousin matching	A relatedMatch B; A exactMatch C; B exactMatch D	C relatedMatch D	× 0.60
5 · Negative pruning	A broader P; B broader Q; $P \cap Q = \emptyset$ (disjoint)	A NOT exactMatch/closeMatch B	Hard constraint
6 · Type correction	Neural: A exactMatch B; Structural: A broader X, B narrower X_eq	Downgrade to A broadMatch B	Symbolic overrides neural

5.2 Implementation

Rules 1, 3, and 5 are the highest-priority implementations for the hackathon — they are self-contained, require only the edge list and hierarchy, and directly address the three main failure modes of string reconciliation.

The full inference graph is built iteratively: apply rules to the aggregated mapping graph, add inferred edges with reduced confidence weights, and repeat (max 2 hops to bound complexity). Inferred edges feed back into GNN training as augmented data.

5.3 SKOS Constraint Post-processing

After neural prediction, logical consistency is enforced:

- exactMatch is symmetric — assert reverse direction automatically
- exactMatch implies closeMatch — add inferred closeMatch edges
- narrowMatch + exactMatch on same pair → keep higher-confidence, flag conflict

- Hierarchy preservation — mapping(A broader B) should satisfy mapping(A) broader-or-equal mapping(B) in target vocabulary

7 · Established Frameworks

6.1 GSAS — Gradual Semantic Alignment for SKOS

GSAS (Thiery et al.) augments SKOS mappings with a normalised degree of connection in $[0, 1]$, expressed via the **SKOSplus ontology**. It defines four complementary alignment models ranging from a minimal SKOS baseline to empirically grounded approaches derived from linguistic probability perceptions.

Fit with this project:

- GSAS provides the formal output format — serialize all produced mappings as SKOSplus RDF with normalized confidence scores.
- The four GSAS models map directly onto the four system tiers: string baseline → bi-encoder → GNN → fine-tuned cross-encoder.
- This project constitutes an empirical validation and neural grounding of the GSAS framework — a citable research contribution.

6.2 AMT — Academic Meta Tool

AMT (mainzed / i3mainz / RGZM) implements weighted graph reasoning over RDF triplestores via a SPARQL endpoint. Edge weights express vagueness — a relationship exists only *to a certain degree*. AMT axioms defined in an OWL ontology drive automatic inference over the weighted graph.

Fit with this project:

- AMT's weighted edge composition directly implements Rules 1–4 from Section 5.1 — transitivity decay, inheritance, etc.
- The SPARQL endpoint makes inferred mappings queryable without custom graph traversal code.
- AMT comes from the same cultural heritage / archaeological informatics community as the target domain (museum terminology alignment).

Practical note: full RDF4J triplestore setup takes ~half a day. For the hackathon, implement equivalent logic in Python but frame as 'AMT-inspired weighted reasoning' and reference their axiom patterns.

8 · Integrated Pipeline

The three layers — neural retrieval, symbolic enrichment, and formal serialization — compose into a single end-to-end alignment pipeline:

Stage	Component	Output
1 · Concept encoding	Concept-as-document builder + multilingual-e5 / BGE-M3	Dense vector per concept (384-768 dim)
2 · GNN refinement	HeteroConv / GraphSAGE over HeteroData mapping graph	Graph-context-aware embedding per concept
3 · Candidate retrieval	FAISS approximate nearest-neighbor index	Top-k candidate pairs per query concept
4 · Reranking	Fine-tuned multilingual cross-encoder	Scored & typed candidates (relation type + confidence)
5 · Symbolic enrichment	AMT-style inference rules (Rules 1–6, Section 5.1)	Additional inferred candidates; type corrections
6 · Constraint enforcement	SKOS logical constraint post-processor	Logically consistent mapping set
7 · Output serialization	SKOSplus / GSAS ontology + rdflib	Valid RDF with normalized confidence scores [0,1]

Output per concept pair: ranked list of candidates with predicted relation type, confidence score [0,1], evidence trail (which rules/models contributed), and human disagreement rate from training data. This is qualitatively richer than any string-matching reconciliation tool.

9 · Two-Day Execution Plan

9.1 Hardware

Resource	Use
Local GPU (12 GB)	GNN training (NeighborLoader mini-batch), cross-encoder fine-tuning with gradient checkpointing
Academic cloud (large LLMs)	Coding assistance (Devstral 100B, Qwen 30B); LLM-in-the-loop disambiguation demo
Google Colab T4/A100	GNN training if local GPU insufficient; upload precomputed embeddings (~500 MB)
CPU machines	Data preprocessing, embedding precomputation (parallelisable), inference rule execution

9.2 Day 1

Time	Task	Deliverable
Morning	Finish mapping download; build aggregated edge list; implement concept-as-document encoder	data/ folder populated; concept embeddings computed
Late morning	Implement Task 1 (terms → thesaurus): cluster with AgglomerativeClustering; serialize as SKOS Turtle	Working demo on 30 Werkblattscience terms
Afternoon	Implement Task 2 (insert into thesaurus) + Task 3 (cross-thesaurus mapping with FAISS)	All three tasks producing output
Evening	SKOS constraint post-processing (Rule 5 + symmetry); string baseline evaluation harness	Baseline hits@1, hits@5, MRR numbers

9.3 Day 2

Time	Task	Deliverable
Morning	Fine-tune sentence transformer on JSKOS pairs (1 epoch, confidence-weighted); rerun evaluation	Improved metrics — quantified gain over baseline
Late morning	Add asymmetric projection heads; train GNN (GraphSAGE, NeighborLoader, 12 GB GPU)	GNN embeddings; further metric improvement
Afternoon	Symbolic inference Rules 1 + 3; LLM-in-the-loop demo for ambiguous cases (Ollama local)	Inferred mappings; LLM reasoning trace
Late afternoon	SKOSplus / GSAS serialization; prepare demo notebook	Presentation-ready demo

10 · Evaluation

Hold out 10% of high-confidence mapping pairs (confidence ≥ 0.8 , $n_concordances \geq 2$) as a clean test set. Evaluate each system tier independently — this gives an ablation study with quantified improvement at each step.

Metric	What it measures	String baseline can compute?
hits@1	Correct match is top-ranked result	Partially (exact only)
hits@5	Correct match in top 5 results	No
MRR	Mean reciprocal rank across all queries	No
type accuracy	Predicted SKOS relation type matches ground truth	No (always exactMatch)
synonym recall	Pairs matched where labels differ	No
false positive rate	Homonym / disambiguation failures	No
inferred coverage	% unmapped pairs covered by symbolic inference	No

Expected narrative: String matching → 40–50% hits@1. Multilingual embeddings → 60–65%. GNN + fine-tuning → 70–75%. Full pipeline with symbolic rules → 75–80%. Each step is a measurable, presentable result.

11 · Related Work

System / Paper	Relevance
BERTMap (Pan et al. 2021) — DeepOnto	Direct predecessor: BERT fine-tuned on ontology alignment using synonyms. <code>pip install deeponeto</code> . Compare against or extend.
LogMap	Strong symbolic baseline. Pure logic-based ontology matching. Establishes how much neural adds.
MELT Evaluation Toolkit	Standard evaluation framework for ontology matching — use for credible metrics.
GSAS (Thiery et al.)	Formal framework for graded SKOS mappings. SKOSplus ontology = your output format. This project empirically grounds their theoretical models.
AMT (mainzed / i3mainz / RGZM)	Weighted graph reasoning over SPARQL. AMT axioms implement your symbolic inference rules. Same cultural heritage domain.

RotatE / ComplEx	Knowledge graph completion (embedding-based). Generic KG completion = your Task 3 without SKOS constraints. Your contribution: SKOS semantics as hard constraints.
HGT — Heterogeneous Graph Transformer	PyTorch Geometric. Best GNN architecture for your multi-relation-type graph. Drop-in upgrade from GraphSAGE baseline.

Core contribution in one sentence: A neuro-symbolic SKOS alignment pipeline trained on 1.1 million confidence-weighted human mapping pairs, producing ranked typed candidates with GSAS-compatible confidence scores — reducing false positives, recovering synonym and cross-language matches, and filling coverage gaps via symbolic inference over the mapping graph.